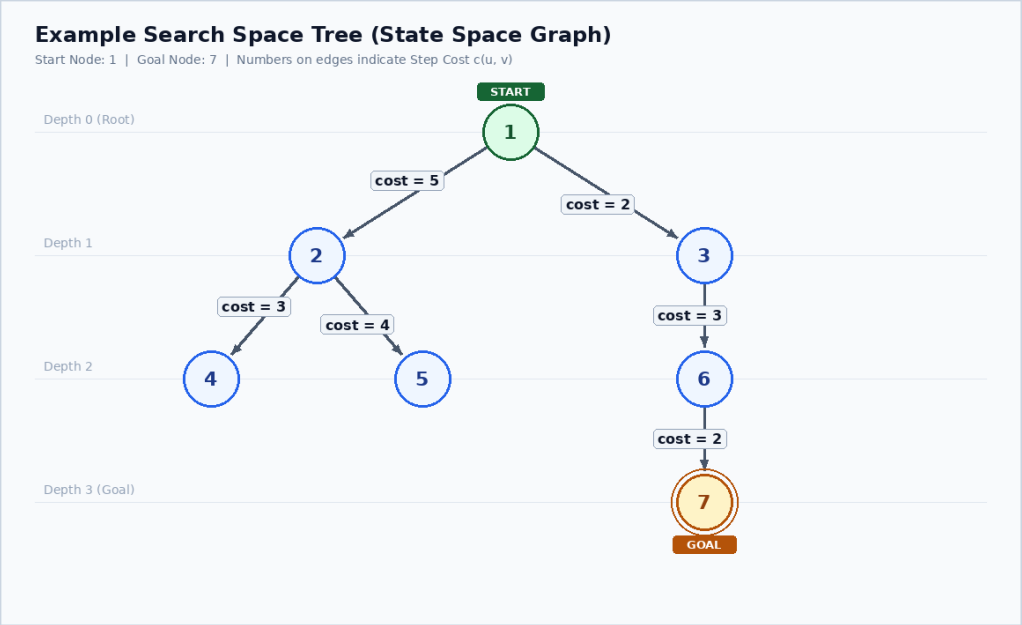
কৃত্রিম বুদ্ধিমত্তা (Artificial Intelligence): আন-ইনফর্মড সার্চ অ্যালগরিদম কমপ্লিট গাইড

CSE 3rd/4th Year Semester Final & Competitive Exam Standard / Full Step-by-Step Trace Tables & Analysis

আন-ইনফর্মড সার্চ (Uninformed / Blind Search) অ্যালগরিদমে লক্ষ্য (Goal) সম্পর্কে কোনো পূর্ব জ্ঞান বা হিউরিস্টিক (Heuristic) তথ্য থাকে না। শুধুমাত্র সমস্যার স্টেট-স্পেস গ্রাফ ও ট্রাভার্সাল রুল ব্যবহার করে লক্ষ্য নোড খুঁজে বের করতে হয়। এই গাইডে ক্লাসিক্যাল ৬টি সার্চ অ্যালগরিদমের মেমোরি ডেটা স্ট্রাকচার, ফ্রিঞ্জ/স্ট্যাক/কিউ স্ট্যাটাস এবং পরীক্ষার খাতায় ফুল মার্কস পাওয়ার উপযোগী নিখুঁত ট্রেস টেবিল উপস্থাপন করা হলো।

১. স্টেট স্পেস গ্রাফ মডেল (Search Space Graph Representation)

পরীক্ষায় অ্যালগরিদমের কার্যপ্রণালী ও পার্থক্য স্পষ্ট করার জন্য নিচের ৭-নোড বিশিষ্ট গ্রাফটি আদর্শ উদাহরণ হিসেবে নেওয়া হলো। এখানে প্রতিটি এজের পাশে নির্দিষ্ট ট্রাভেল কস্ট (Step Cost) দেওয়া হয়েছে, যাতে Uniform-Cost Search (UCS)-এর আসল বৈশিষ্ট্য সাধারণ BFS থেকে স্পষ্টভাবে দৃশ্যমান হয়।



চিত্র ১: পরীক্ষামূলক স্টেট স্পেস ট্রি *(Start Node = ১, Goal Node = ৭, এজের সংখ্যা = ট্রাভেল কস্ট)*

- নোড সেট **(Nodes):** V = {১, ২, ৩, ৪, ৫, ৬, ৭} | স্টার্ট নোড: ১ | লক্ষ্য/গোল নোড: ৭
- এজ ও স্টেপ কস্ট: (১ → ২: ৫), (১ → ৩: ২), (২ → ৪: ৩), (২ → ৫: ৪), (৩ → ৬: ৩), (৬ → ৭: ২)
- টাই-ব্রেকিং নীতি **(Tie-breaking rule):** যদি ফ্রন্টিয়ারে একাধিক নোডের অগ্রাধিকার একই হয়, তবে ছোট নোড আইডি (বাম দিকের নোড) আগে এক্সপ্যান্ড হবে।

২. Breadth-First Search (BFS)

BFS হলো লেভেল-বাই-লেভেল ট্রাভার্সাল অ্যালগরিদম। এটি রুট থেকে শুরু করে প্রতিটি ডেপথের সকল নোডকে ক্রমান্বয়ে এক্সপ্লোর করে পরবর্তী ডেপথে প্রবেশ করে।

- ব্যবহৃত ডেটা স্ট্রাকচার: FIFO Queue (First In First Out)।
- গোল টেস্ট **(Goal Test):** স্ট্যান্ডার্ড থিওরিতে (Russell & Norvig) নোড জেনারেশনের সময়ই গোল টেস্ট করা যায়, তবে পরীক্ষার খাতায় টেবিল প্রদর্শনের ক্ষেত্রে এক্সপানশনের সময় দেখানো হয়।
- ট্রেসিং নিয়ম: Queue-এর বাম পাশ Front (পপ হবে) এবং ডান পাশ Rear (নতুন চাইল্ড পুশ হবে)।

Step	Expanded Node	Generated Children	Queue State (Front → Rear)	Visited / Closed Set	Goal Check?
Init	-	-	[১]	{}	No
১	১	২, ৩	[২, ৩]	{১}	No
২	২	৪, ৫	[৩, ৪, ৫]	{১, ২}	No
৩	৩	৬	[৪, ৫, ৬]	{১, ২, ৩}	No
৪	৪	None	[৫, ৬]	{১, ২, ৩, ৪}	No
৫	৫	None	[৬]	{১, ২, ৩, ৪, ৫}	No
৬	৬	৭	[৭]	{১, ২, ৩, ৪, ৫, ৬}	No
৭	৭	None	[] (Empty)	{১, ২, ৩, ৪, ৫, ৬, ৭}	Goal Found! STOP

BFS এর ফলাফল ও তাত্ত্বিক জটিলতা বিশ্লেষণ

• নোড এক্সপানশন ক্রম (Traversal Order): ১ → ২ → ৩ → ৪ → ৫ → ৬ → ৭

• প্রাপ্ত পাথ (Solution Path): ১ → ৩ → ৬ → ৭ (পাথ লেন্থ = ৩ স্টেপ, মোট কস্ট = ২ + ৩ + ২ = ৭)

• টাইম কমপ্লেক্সিটি: O(b^d) | স্পেস কমপ্লেক্সিটি: O(b^d) [যেখানে b = ব্রাঞ্চিং ফ্যাক্টর, d = গোলের গভীরতা]

• কমপ্লিটনেস: হ্যাঁ (যদি ব্রাঞ্চিং ফ্যাক্টর b সসীম হয়) | অপটিমালিটি: হ্যাঁ (যদি প্রতিটি এজের কস্ট সমান হয়)। নন-ইউনিফর্ম কস্টে BFS অপটিমাল পাথ গ্যারান্টি দেয় না।

৩. Depth-First Search (DFS)

DFS একটি নির্দিষ্ট শাখার একদম শেষ নোড (গভীরে) পর্যন্ত এক্সপ্লোর করে এবং ডেড-এন্ডে পৌঁছালে পেছনে ব্যাকট্র্যাক করে পরবর্তী শাখায় যায়।

- ব্যবহৃত ডেটা স্ট্রাকচার: LIFO Stack (Last In First Out)।
- স্ট্যাক পুশ অর্ডার কৌশল: বামদিকের নোডকে আগে ভিজিট করতে চাইলে স্ট্যাকে ডানদিকের চাইল্ড আগে এবং বাঁদিকের চাইল্ড পরে পুশ করতে হবে। নিচের টেবিলে স্ট্যাকের ডান প্রান্তকে 'Top' ধরা হয়েছে।

Step	Popped Node	Children Generated	Stack State (Top is at Right)	Visited / Closed Set	Goal Check?
Init	-	-	[১] (Top=১)	{}	No
১	১	২, ৩ (পুশ: ৩, ২)	[৩, ২] (Top=২)	{১}	No
২	২	৪, ৫ (পুশ: ৫, ৪)	[৩, ৫, ৪] (Top=৪)	{১, ২}	No
৩	৪	None	[৩, ৫] (Top=৫)	{১, ২, ৪}	No
৪	৫	None	[৩] (Top=৩)	{১, ২, ৪, ৫}	No
৫	৩	৬ (পুশ: ৬)	[৬] (Top=৬)	{১, ২, ৪, ৫, ৩}	No
৬	৬	৭ (পুশ: ৭)	[৭] (Top=৭)	{১, ২, ৪, ৫, ৩, ৬}	No
৭	৭	None	[] (Empty)	{১, ২, ৪, ৫, ৩, ৬, ৭}	Goal Found! STOP

DFS এর ফলাফল ও তাত্ত্বিক জটিলতা বিশ্লেষণ

• নোড এক্সপানশন ক্রম: ১ → ২ → ৪ → ৫ → ৩ → ৬ → ৭

• প্রাপ্ত পাথ: ১ → ৩ → ৬ → ৭

• টাইম কমপ্লেক্সিটি: O(b^m) | স্পেস কমপ্লেক্সিটি: O(b · m) [যেখানে m = গ্রাফের সর্বোচ্চ গভীরতা]

• কমপ্লিটনেস: না (ইনফিনিট ব্রাঞ্চ বা সাইকেল থাকলে আটকে যায়) | অপটিমালিটি: না (যে পাথ আগে পায় সেটিই রিটার্ন করে, কম খরচের পাথ খোঁজে না)।

• প্রধান সুবিধা: অত্যন্ত কম মেমোরি প্রয়োজন (লিনিয়ার স্পেস কমপ্লেক্সিটি)।

৪. Uniform-Cost Search (UCS / Dijkstra's Algorithm)

UCS সর্বদা স্টার্ট নোড থেকে সর্বনিম্ন পুঞ্জীভূত পথ-খরচ (Cumulative Path Cost g(n)) বিশিষ্ট নোডকে অগ্রাধিকার দিয়ে সম্প্রসারিত করে।

- ব্যবহৃত ডেটা স্ট্রাকচার: Priority Queue (পাথ কস্ট g(n) অনুযায়ী এসেন্ডিং অর্ডারে সাজানো)।
- গোল টেস্টের অতি গুরুত্বপূর্ণ নিয়ম: UCS-এ গোল টেস্ট অবশ্যই নোডটি প্রায়োরিটি কিউ থেকে POP করার সময় করতে হবে, জেনারেট হওয়ার সময় নয়! কারণ জেনারেশনের সময় কোনো গোল নোড বেশি কস্ট নিয়ে কিউতে ঢুকলেও পরে বিকল্প সস্তা পাথে আরও কম খরচে পৌঁছানো সম্ভব হতে পারে।

Step	Popped (Node, g)	Generated Children with g(n)	Priority Queue State [(Node, g)]	Visited Set	Goal Check?
Init	**-**	-	[(১, g=০)]	{}	No
১	**(১, g=০)**	২: g=০+৫=৫, ৩: g=০+২=২	[(৩, ২), (২, ৫)]	{১}	No
২	**(৩, g=২)**	৬: g=২+৩=৫	[(২, ৫), (৬, ৫)]	{১, ৩}	No (কস্ট ৫<২)
৩	**(২, g=৫)**	৪: g=৫+৩=৮, ৫: g=৫+৪=৯	[(৬, ৫), (৪, ৮), (৫, ৯)]	{১, ৩, ২}	No (টাই-ব্রেকার ২)
৪	**(৬, g=৫)**	৭: g=৫+২=৭	[(৭, ৭), (৪, ৮), (৫, ৯)]	{১, ৩, ২, ৬}	No
৫	**(৭, g=৭)**	None (Goal Node Popped)	[(৪, ৮), (৫, ৯)]	{১, ৩, ২, ৬, ৭}	Goal Popped! STOP

UCS এর বুদ্ধিমত্তা ও প্রুনিং (Pruning) পাওয়ার

১. প্রুনিং শক্তি: দেখুন, নোড ৪ (কস্ট ৮) এবং নোড ৫ (কস্ট ৯) প্রায়োরিটি কিউতে থাকা সত্ত্বেও কখনোই EXPAND হয়নি! কারণ তাদের চেয়ে কম কস্টে (৭) গোল নোডটি কিউ থেকে পপ হয়ে গেছে।

২. এক্সপানশন ক্রম: ১ → ৩ → ২ → ৬ → ৭ (BFS ও DFS থেকে সম্পূর্ণ আলাদা!)

৩. অপটিমাল পাথ: ১ → ৩ → ৬ → ৭ | মোট ন্যূনতম খরচ = ৭

৪. কমপ্লেক্সিটি: টাইম ও স্পেস = O(b^(1 + floor(C*/epsilon))) [C* = অপটিমাল কস্ট, epsilon = ক্ষুদ্রতম এজ কস্ট]

৫. কমপ্লিটনেস ও অপটিমালিটি: উভয়ই হ্যাঁ (যদি সকল এজের কস্ট epsilon > 0 হয়)।

৫. Depth-Limited Search (DLS)

DLS হলো DFS-এর একটি পরিমার্জিত রূপ, যেখানে ইনফিনিট লুপের বিপদ এড়াতে একটি পূর্বনির্ধারিত গভীরতা সীমা (Depth Limit l) বেঁধে দেওয়া হয়। সীমার বাইরের কোনো চাইল্ড নোড তৈরি করা হয় না। গোল নোড ৭ রয়েছে Depth ৩-এ।

- মূল বৈশিষ্ট্য: যদি Limit l < d (গোল গভীরতা) হয়, তবে সার্চ Cutoff Failure দেয়। আর যদি l >= d হয়, তবে সমাধান পাওয়া যায়।
- ট্রেস তুলনা: Limit l = ২ (ব্যর্থ) এবং Limit l = ৩ (সফল)-এর প্রতিটি ধাপ পাশাপাশি দেখানো হলো:

Step	Expanded (Depth)	Limit l = ২ Action & Stack	Limit l = ৩ Action & Stack	Goal Status & Remarks
১	১ (d=০)	Stack: [৩, ২] (d=১ <= ২)	Stack: [৩, ২] (d=১ <= ৩)	উভয় ক্ষেত্রেই সীমার ভেতর
২	২ (d=১)	Stack: [৩, ৫, ৪] (d=২ <= ২)	Stack: [৩, ৫, ৪] (d=২ <= ৩)	উভয় ক্ষেত্রেই সীমার ভেতর
৩	৪ (d=২)	Limit ২ পৌঁছেছে! চাইল্ড যাবে না	Leaf Node (কোনো চাইল্ড নেই)	l=২ এর সীমা শেষ
৪	৫ (d=২)	Limit ২ পৌঁছেছে! চাইল্ড যাবে না	Leaf Node (কোনো চাইল্ড নেই)	l=২ এর সীমা শেষ
৫	৩ (d=১)	Stack: [৬] (d=২ <= ২)	Stack: [৬] (d=২ <= ৩)	উভয় ক্ষেত্রেই সীমার ভেতর
৬	৬ (d=২)	Limit ২ শেষ! চাইল্ড ৭ (d=৩) বাদ!	Stack: [৭] (d=৩ <= ৩ পুশ)	মূল পার্থক্য! l=২ তে ৭ বাদ পড়েছে
৭	৭ (d=৩)	Not Reached! (Cutoff Failure)	Goal Node Popped! [SUCCESS]	l=২ ব্যর্থ, কিন্তু l=৩ সফল!

DLS এর ফলাফল ও সীমাবদ্ধতা বিশ্লেষণ

• কেস ১ (Limit l = ২): এক্সপানশন ক্রম: ১ → ২ → ৪ → ৫ → ৩ → ৬। ফলাফল: Cutoff Failure (৭ বাদ পড়েছে)।

• কেস ২ (Limit l = ৩): এক্সপানশন ক্রম: ১ → ২ → ৪ → ৫ → ৩ → ৬ → ৭। ফলাফল: Goal Found (Success)।

• কমপ্লেক্সিটি: টাইম = O(b^l) | স্পেস = O(b · l) | কমপ্লিটনেস: না (যদি l < d হয়) | অপটিমালিটি: না।

৬. Iterative Deepening Search (IDS / IDDFS)

IDS হলো আন-ইনফর্মড সার্চের সবচেয়ে সুষম অ্যালগরিদম। এটি DFS-এর মেমোরি দক্ষতা (Linear Space) এবং BFS-এর কমপ্লিটনেস ও অপটিমালিটিকে একত্র করে। এটি ক্রমান্বয়ে Limit = ০, ১, ২, ৩ ... বাড়িয়ে DLS রান করে যতক্ষণ না গোল পাওয়া যায়।

Iteration Limit	Depth Boundary	Nodes Expanded in this Iteration	Total Nodes Expanded	Status
Limit = ০	**Depth** ০	১	১	Cutoff (Goal not found)
Limit = ১	**Depth** ০ থেকে ১	১ → ২ → ৩	৩	Cutoff (Goal not found)
Limit = ২	**Depth** ০ থেকে ২	১ → ২ → ৪ → ৫ → ৩ → ৬	৬	Cutoff (Goal not found)
Limit = ৩	**Depth** ০ থেকে ৩	১ → ২ → ৪ → ৫ → ৩ → ৬ → ৭	৭	Goal 7 Found! SUCCESS

IDS এর মেমোরি ও ওভারহেড সংক্রান্ত গাণিতিক ব্যাখ্যা (Exam Favorite Question)

• শিক্ষার্থীদের সাধারণ ভুল ধারণা: বারবার আগের লেভেল এক্সপ্যান্ড করায় IDS অনেক বেশি সময় নষ্ট করে।

• গাণিতিক সত্য: সার্চ ট্রির নোড সংখ্যা জ্যামিতিক হারে (Exponentially) বাড়ে। রুট নোড মাত্র d বার তৈরি হয়, কিন্তু সবচেয়ে নিচের লেভেলের বিশাল সংখ্যক নোড তৈরি হয় মাত্র ১ বার।

• মোট নোড এক্সপানশন = (d+১)·১ + d·b + (d-১)·b^২ + ... + ১·b^d = O(b^d)।

• টাইম কমপ্লেক্সিটি: O(b^d) [BFS-এর সমান] | স্পেস কমপ্লেক্সিটি: O(b · d) [DFS-এর মতো ক্ষুদ্র! মেমোরি কখনো ক্র্যাশ করে না]।

৭. Bidirectional Search (BDS)

BDS একই সাথে দুটি সমান্তরাল সার্চ পরিচালনা করে: একটি স্টার্ট নোড থেকে সামনের দিকে (Forward Search), অন্যটি গোল নোড থেকে পেছনের দিকে (Backward Search)। যখন উভয়ের ফ্রন্টিয়ার কোনো সাধারণ নোডে মিলিত হয়, তখন একটি পূর্ণাঙ্গ পাথ গঠিত হয়।

Direction	Current Expanded	Frontier Nodes (Queue State)	Intersection Check	Status
Forward (Start: ১)	১	{২, ৩}	Backward-এর সাথে কোনো মিল নেই	In Progress
Backward (Goal: ৭)	৭ **(Predecessor: ৬)**	{৬}	Forward-এর সাথে কোনো মিল নেই	In Progress
Forward	৩ (চাইল্ড: ৬)	{২, ৬}	উভয়ের ফ্রন্টিয়ারে নোড ৬ বর্তমান!	Intersection at Node ৬!
Path Construction	**Forward:** ১→৩→৬	Backward: ৬→৭	পূর্ণাঙ্গ পাথ: ১ → ৩ → ৬ → ৭	Search Completed!

BDS এর কার্যকারিতা ও সীমাবদ্ধতা

• গাণিতিক সুবিধা: O(b^d) এর পরিবর্তে টাইম কমপ্লেক্সিটি দাঁড়ায় O(2 · b^(d/2)) = O(b^(d/2))। উদাহরণস্বরূপ, b=১০ এবং d=৬ হলে ট্র্যাডিশনাল সার্চে ১০^৬ = ১০,০০,০০০ নোড এক্সপ্যান্ড হতো, যেখানে BDS-এ মাত্র ২ × ১০^৩ = ২,০০০ নোড এক্সপ্যান্ড হবে!

• প্রধান সীমাবদ্ধতা: পেছনের দিকে সার্চ করার জন্য গোল স্টেট সুনির্দিষ্ট ও অনন্য হতে হয় এবং প্রতিটি এজের বিপরীতমুখী পথ (Predecessor) জানা থাকা আবশ্যক।

৮. সার্বিক তুলনামূলক ম্যাট্রিক্স (Uninformed Search Master Matrix)

বিশ্ববিদ্যালয় ফাইনাল পরীক্ষার খাতায় নিচের ছকটি হুবহু লিখলে সর্বোচ্চ পূর্ণ নম্বর নিশ্চিত করা যায়:

Algorithm	Data Structure	Time Complexity	Space Complexity	Complete?	Optimal?
BFS	**FIFO Queue**	O(b^d)	O(b^d)	Yes (যদি b সসীম)	Yes (যদি কস্ট সমান)
DFS	**LIFO Stack**	O(b^m)	O(b · m)	No (লুপ থাকলে আটকে যায়)	No
UCS	**Priority Queue**	O(b^(1 + floor(C*/eps)))	O(b^(1 + floor(C*/eps)))	Yes (যদি কস্ট > 0)	Yes (সর্বদা কস্ট-অপটিমাল)
DLS	**Stack + Limit l**	O(b^l)	O(b · l)	No (যদি l < d)	No
IDS	**Iterative Stack**	O(b^d)	O(b · d)	Yes (যদি b সসীম)	Yes (যদি কস্ট সমান)
BDS	**Two Queues**	O(b^(d/2))	O(b^(d/2))	Yes (যদি b সসীম)	Yes (যদি কস্ট সমান)

৯. পরীক্ষার হলের বিশেষ সতর্কতা ও ফুল মার্কস সিক্রেট (Exam Survival Tips)

> পরীক্ষার খাতায় যেসব ভুলের কারণে নম্বর কাটা যায়
>
> ১. UCS-এ গোল টেস্ট জেনারেশনে দেখানো: এটি AI পরীক্ষার এক নম্বর কমন ভুল। UCS-এ কোনো নোড জেনারেট হওয়ার সাথে সাথে গোল বলা যাবে না; নোডটি যখন সর্বনিম্ন খরচের হিসেবে পপ (Pop) হবে তখনই কেবল গোল ঘোষণা করতে হবে।
>
> ২. স্ট্যাকের Top কোন দিকে তা উল্লেখ না করা: DFS ও DLS লেখার সময় স্ট্যাকের কোন দিকটা Top (বাম না ডান) তা টেবিলে বা নোট আকারে অবশ্যই লিখে দিতে হবে।
>
> ৩. ট্রি সার্চ বনাম গ্রাফ সার্চ গুলিয়ে ফেলা: গ্রাফে যদি সাইকেল থাকে, তবে Visited Set / Closed List বজায় না রাখলে DFS ও BFS উভয়ই অনন্ত লুপে আটকাতে পারে। তাই Visited Set কলাম প্রদর্শন করা অত্যন্ত জরুরি।
>
> ৪. IDS-এর স্পেস কমপ্লেক্সিটি নিয়ে ভুল ধারণা: IDS-এর স্পেস কখনোই O(b^d) নয়, এটি O(b · d); কারণ এটি যেকোনো মুহূর্তে শুধুমাত্র একটি শাখার নোডই মেমোরিতে রাখে।